

COMMON COURSE ASSIGNMENT

CSA0404 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

NAME BP Venkata Uday Kumar

Reg NO: 192411203

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	21/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: “OSLearn” – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).
- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.
- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.

- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

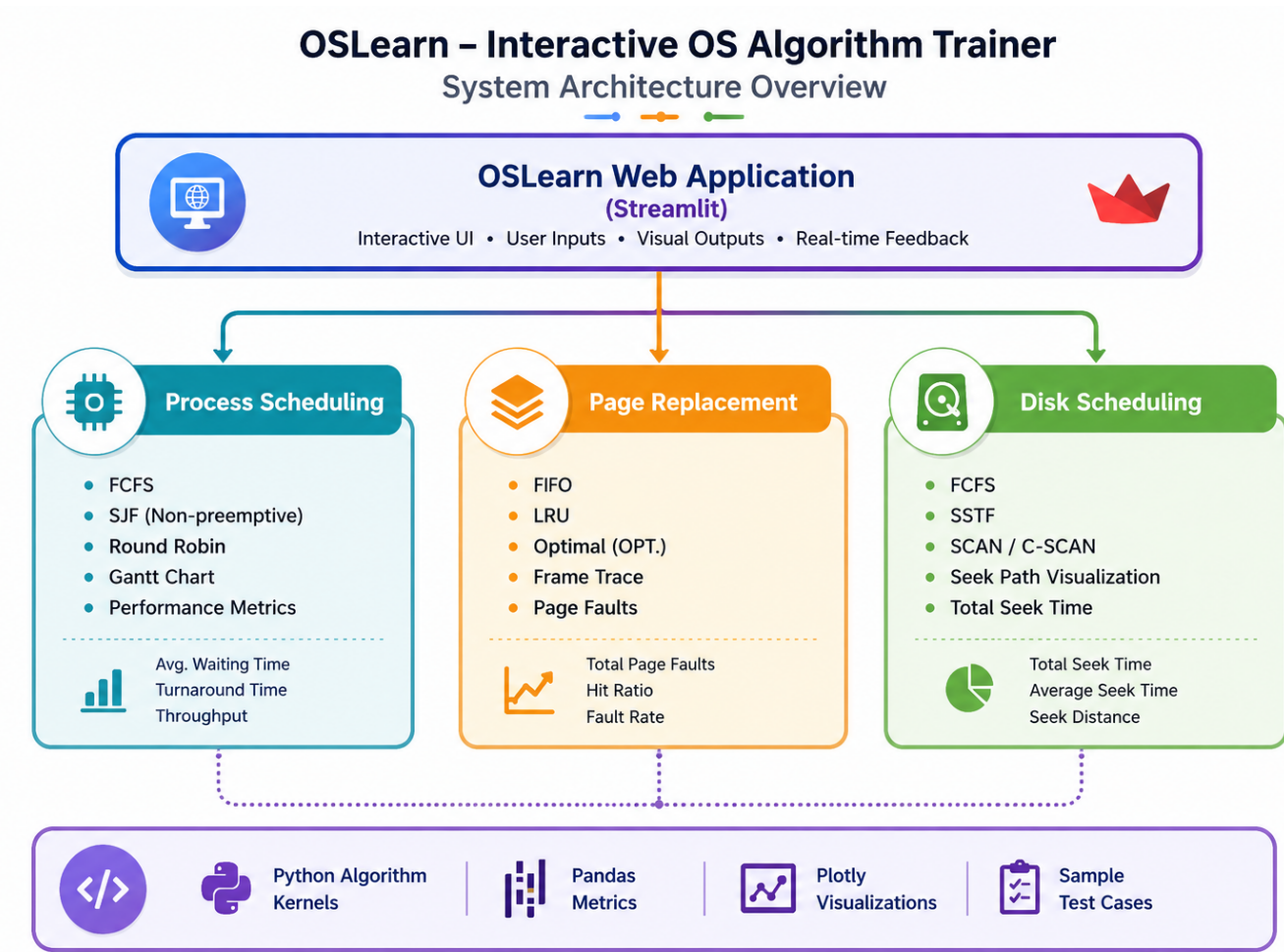
1. Problem Understanding and Formulation

1.1 Context & Problem Restatement

Operating System (OS) concepts—specifically CPU Process Scheduling, Memory Management (Page Replacement), and Disk I/O Scheduling—are fundamentally dynamic and state-dependent. Traditional didactic instruction relies on static textbook diagrams and manual trace tables, which often fail to convey runtime state transitions and algorithmic trade-offs.

OSLearn is an interactive, unified educational software system designed to bridge the gap between theoretical OS principles and practical comprehension. The system enables students to simulate, visualize, and benchmark foundational OS algorithms across three core resource management domains through an interactive web interface built with Streamlit.

Figure 1: OSLearn System Architecture Overview



1.2 Specific OS Challenges Modeled

Domain	Underlying OS Problem	Algorithmic Challenge
Process Management (CO2)	Multi-process CPU multiplexing & starvation	Trade-offs: throughput, TAT, WT under preemption vs non-preemption
Memory Management (CO3)	Limited physical frames under demand paging	Minimize page-fault frequency; avoid thrashing (FIFO vs LRU vs Optimal)
Secondary Storage (CO4)	High mechanical latency of magnetic disk I/O	Minimize total cylinder seek distance; prevent starvation

1.3 Expected Outcomes

- Deterministic Execution: Identical inputs produce identical, verifiable traces.
- Step-by-Step Transparency: Intermediate states (ready queue, frame snapshots, seek transitions) are exposed.

- Comparative Analytics: Quantitative metrics (WT, TAT, Fault Count, Seek Distance) side-by-side.
- Clean Streamlit UI with separate modules and sample test cases.

1.4 Assumptions & System Model

Parameter	Value	Justification
Physical Memory	8 GB	Matches modern x86/ARM paging hardware
Page / Frame Size	4 KB	Standard hardware page size (2^{12} bytes)
Logical Address Space	64 MB	Typical multi-threaded educational process footprint
Disk Geometry	200 cylinders (0–199)	Standard academic baseline for visualization
Context Switch Overhead	0 (idealized)	Focus on pure algorithmic comparison

1.5 Constraints Satisfaction (Section D)

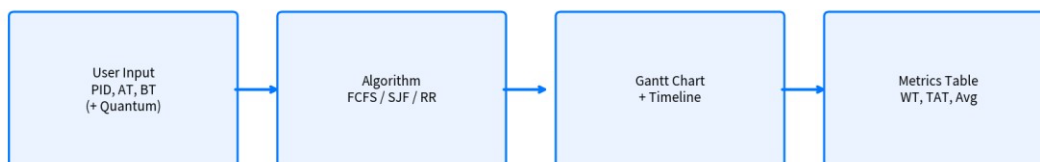
- Technology Stack: Pure Python 3 + Streamlit + Pandas + Plotly. No database, no external LLM.
- Interface: Clean multipage Streamlit app with separate sections for each module.
- Reliability: Correct, reproducible algorithms; transparent intermediate steps; edge-case handling.
- Responsible Computing: Transparency, reproducibility, educational ethics.

2. Application of Course Knowledge

2.1 Process Scheduling Algorithms

Three classic CPU scheduling algorithms are implemented: FCFS, SJF (Non-preemptive), and Round Robin. Users enter Process ID, Arrival Time, Burst Time, and (for RR) Time Quantum. The system produces Gantt-chart timelines and tables of Waiting Time, Turnaround Time, and averages for side-by-side comparison. **Figure 2: Process Scheduling Module Data Flow**

Process Scheduling Module – Data Flow



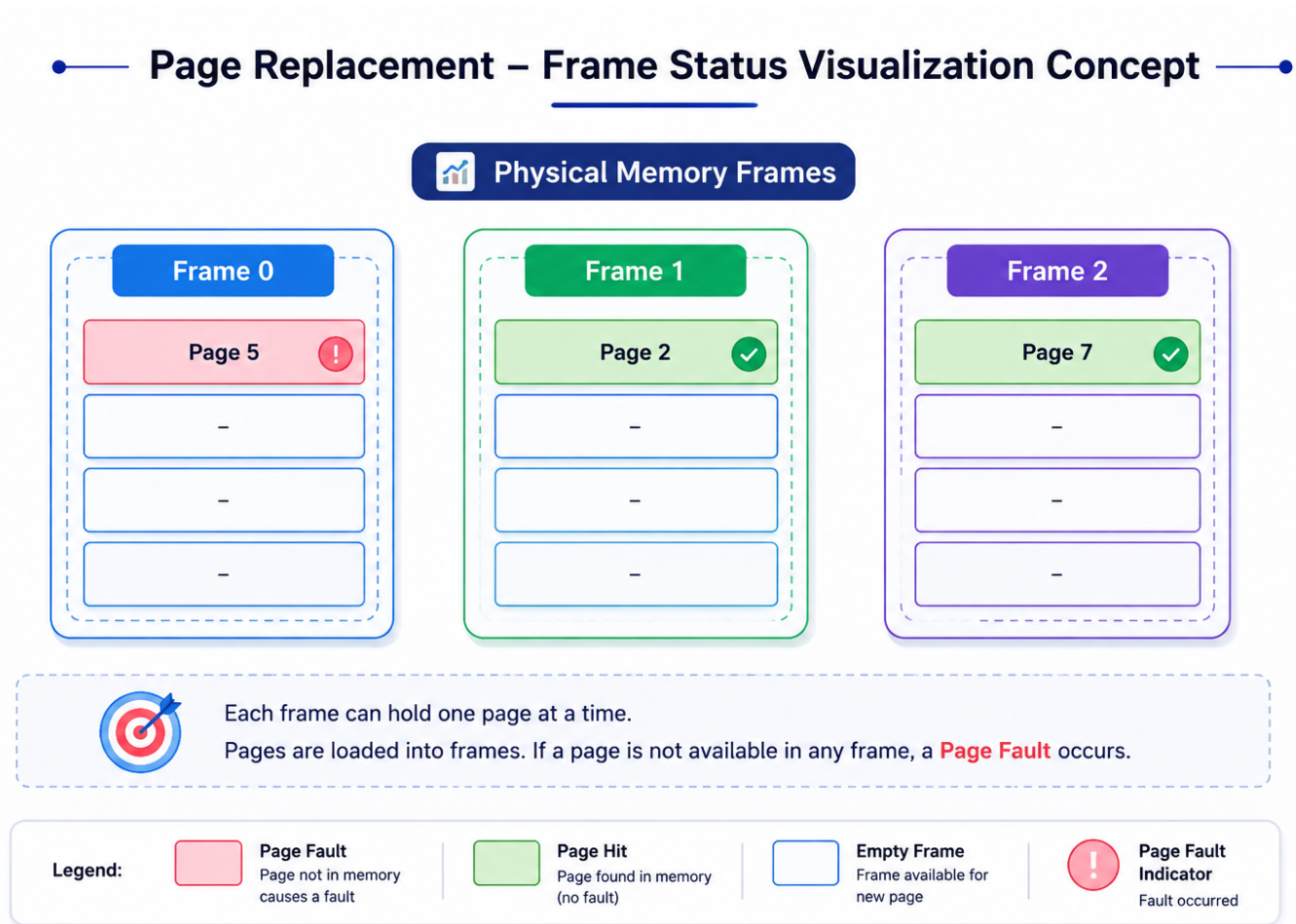
2.2 Memory Management – Page Replacement

FIFO and LRU (with Optimal as benchmark) are implemented. Users supply the number of frames and a pagereference string. The system shows step-by-step frame status after each reference, highlights page faults, and reports total page-fault counts for comparison.

Paging Architecture Calculations (from assignment assumptions):

- Physical frames = 8 GB / 4 KB = $2^{21} = 2,097,152$ frames
- Logical pages for 64 MB process = 64 MB / 4 KB = $2^{14} = 16,384$ pages
- VPN width = 14 bits; Offset = 12 bits; PFN width = 21 bits
- Single-level page table size $\approx 16,384 \times 4$ bytes = 64 KB

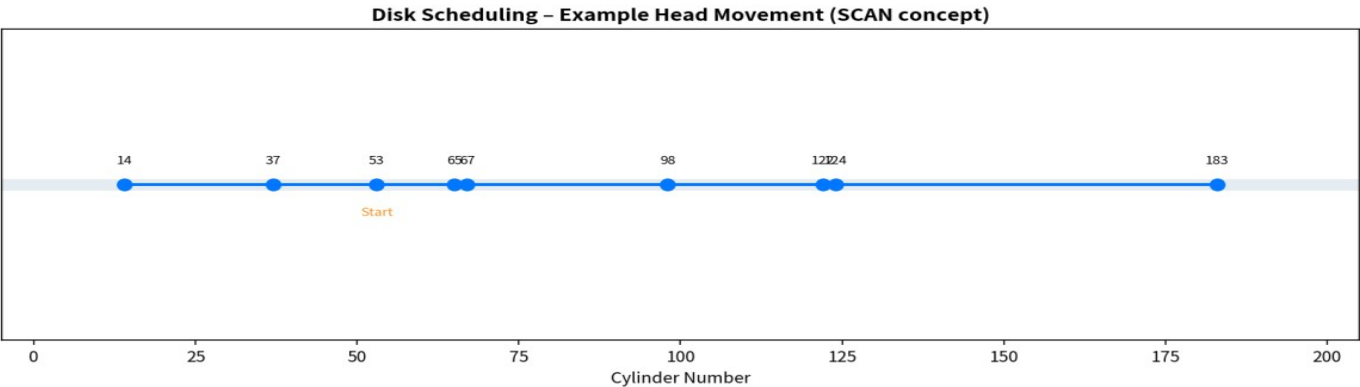
Figure 3: Page Replacement Frame-Status Concept



2.3 Disk Scheduling

FCFS, SSTF and SCAN (LOOK variant) are implemented. Users enter the disk request queue and initial head position. The system displays the service order (seek sequence) and total seek time for each algorithm, enabling clear comparative analysis.

Figure 4: Disk Head Movement Concept (SCAN)



3. Solution / Design / Methodology

3.1 Software Architecture

- Presentation Layer: Streamlit multipage engine (app.py + pages/)
- Visualization Layer: Plotly Gantt, HTML frame matrix, seek-path charts
- Algorithm Kernel: scheduling.py, page_replacement.py, disk_scheduling.py
- Sample Repository: Pre-calibrated presets for quick testing

3.2 End-to-End Workflow

User selects module → enters parameters or loads sample → Input Validation → Algorithm Kernel computes traces & metrics → Visual Component Renderer (Gantt / Frame Grid / Seek Curve) + Metrics Engine → Streamlit renders Individual + Compare-All tabs.

4. Use of Modern Tools

Layer	Tool / Library
Language / Runtime	Python 3.x
Web Framework	Streamlit
Data Processing	Pandas / NumPy
Visualization	Plotly
Version Control	Git & GitHub

Core algorithms (FCFS, SJF, Round Robin, FIFO, LRU, FCFS/SSTF/SCAN disk) are implemented in pure Python with clear intermediate state tracking so every step is transparent and reproducible. No external LLM or database is required.

5. Results and Validation

5.1 Process Scheduling – Sample Results

Test Case: P1(AT=0,BT=8), P2(AT=1,BT=4), P3(AT=2,BT=9), P4(AT=3,BT=5); Quantum q=3

Algorithm	Mean Waiting Time	Mean Turnaround Time	Notes
FCFS	9.75	16.25	Simple; convoy-effect risk
SJF (Non-preemptive)	6.50	13.00	Optimal mean WT/TAT
Round Robin (Q=3)	9.00	15.50	Fair, starvation-free

5.2 Page Replacement – Sample Results

Reference string $\sigma = \langle 1,2,3,4,1,2,5,1,2,3,4,5 \rangle$ with 3 frames; second case with 4 frames.

Algorithm	Test Case 1 (3 frames)	Test Case 2 (4 frames)
FIFO	9 faults (75.0%)	7 faults (53.8%)
LRU	10 faults (83.3%)	6 faults (46.2%)
Optimal (benchmark)	7 faults (58.3%)	5 faults (38.5%)

5.3 Disk Scheduling – Sample Results

Cylinders 0–199, Initial head = 53, Queue = [98, 183, 37, 122, 14, 124, 65, 67]

Algorithm	Total Seek Distance	Starvation-Free?	Fairness
FCFS	640 cylinders	Yes	Poor
SSTF	236 cylinders	No (risk at ends)	Uneven
SCAN (LOOK)	299 cylinders	Yes	High
C-SCAN (C-LOOK)	322 cylinders	Yes	Best uniform

6. Analysis and Engineering Decisions

6.1 Process Scheduling Trade-offs

- FCFS suffers from the convoy effect when long bursts delay short jobs.
- SJF minimizes average waiting time when burst times are known; may starve long jobs.
- Round Robin bounds response time ($RT \leq (n-1) \cdot q$); quantum choice balances fairness vs overhead.

6.2 Page Replacement Trade-offs

- FIFO has low overhead but can exhibit Belady's anomaly.
- LRU exploits temporal locality and avoids Belady's anomaly; practical approximations (Clock) exist.
- Optimal is the theoretical lower bound but requires future knowledge.

6.3 Disk Scheduling Trade-offs

- SSTF yields lowest seek distance but can starve outer-track requests.
- SCAN / C-SCAN trade a modest increase in total seek for bounded waiting time and fairness.


```

disk_scheduling.py
algorithms > disk_scheduling.py > scan
1
2 def fcfs_disk(requests, head):
3     """Serve requests in the order they arrive."""
4     sequence = [head] + list(requests)
5     total = sum(abs(sequence[i + 1] - sequence[i]) for i in range(len(sequence) - 1))
6     return sequence, total
7
8
9 def sstf(requests, head):
10    """Shortest Seek Time First [] always service the nearest pending request."""
11    remaining = list(requests)
12    sequence = [head]
13    current = head
14
15    while remaining:
16        closest = min(remaining, key=lambda x: abs(x - current))
17        sequence.append(closest)
18        current = closest
19        remaining.remove(closest)
20
21    total = sum(abs(sequence[i + 1] - sequence[i]) for i in range(len(sequence) - 1))
22    return sequence, total
23
24
25 def scan(requests, head, direction="right"):
26    Ctrl+L for Agent
27    remaining = sorted(requests)
28    sequence = [head]
29
30    if direction == "right":
31        right = [r for r in remaining if r >= head]
32        left = sorted([r for r in remaining if r < head], reverse=True)
33        for r in right:
34            sequence.append(r)
35        for r in left:
36            sequence.append(r)
37    else: # left

```

```
disk_scheduling.py • page_replacement.py •
algorithms > page_replacement.py > ...

1 Ctrl+L for Agent
2 def fifo(pages, num_frames):
3     """First-In First-Out page replacement."""
4     frames = [None] * num_frames
5     pointer = 0
6     loaded = set()
7     steps = []
8     faults = []
9
10    for page in pages:
11        if page in loaded:
12            faults.append(False)
13        else:
14            faults.append(True)
15            if frames[pointer] is not None:
16                loaded.discard(frames[pointer])
17            frames[pointer] = page
18            loaded.add(page)
19            pointer = (pointer + 1) % num_frames
20
21        steps.append(frames[:])
22
23    return steps, faults, sum(faults)
24
25
26 def lru(pages, num_frames):
27     """Least Recently Used page replacement."""
28     frames = [None] * num_frames # fixed frame slots
29     page_slot = {} # page -> slot index
30     time_used = {} # page -> last access timestamp
31     next_empty = 0 # pointer to next empty slot (during fill-up phase)
32     steps = []
33     faults = []
34
35     for t, page in enumerate(pages):
36         if page in page_slot:
37             # Hit - update recency
```

```
disk_scheduling.py • page_replacement.py • scheduling.py •
algorithms > scheduling.py > ...
1 Ctrl+L for Agent
2 import pandas as pd
3
4
5 def fcfs(processes):
6     """First Come First Served scheduling (non-preemptive)."""
7     procs = sorted(processes, key=lambda x: (x["arrival"], x["pid"]))
8     current_time = 0
9     gantt_blocks = []
10    results = []
11
12    for p in procs:
13        if current_time < p["arrival"]:
14            current_time = p["arrival"]
15        start = current_time
16        end = current_time + p["burst"]
17        gantt_blocks.append({"pid": p["pid"], "start": start, "end": end})
18        wt = start - p["arrival"]
19        tat = end - p["arrival"]
20        results.append(
21            {
22                "Process": p["pid"],
23                "Arrival": p["arrival"],
24                "Burst": p["burst"],
25                "Start": start,
26                "Finish": end,
27                "Waiting Time": wt,
28                "Turnaround Time": tat,
29            }
30        )
31        current_time = end
32
33    df = pd.DataFrame(results)
34    return gantt_blocks, df
35
36
37 def sjf(processes):
```

7. Broader Considerations

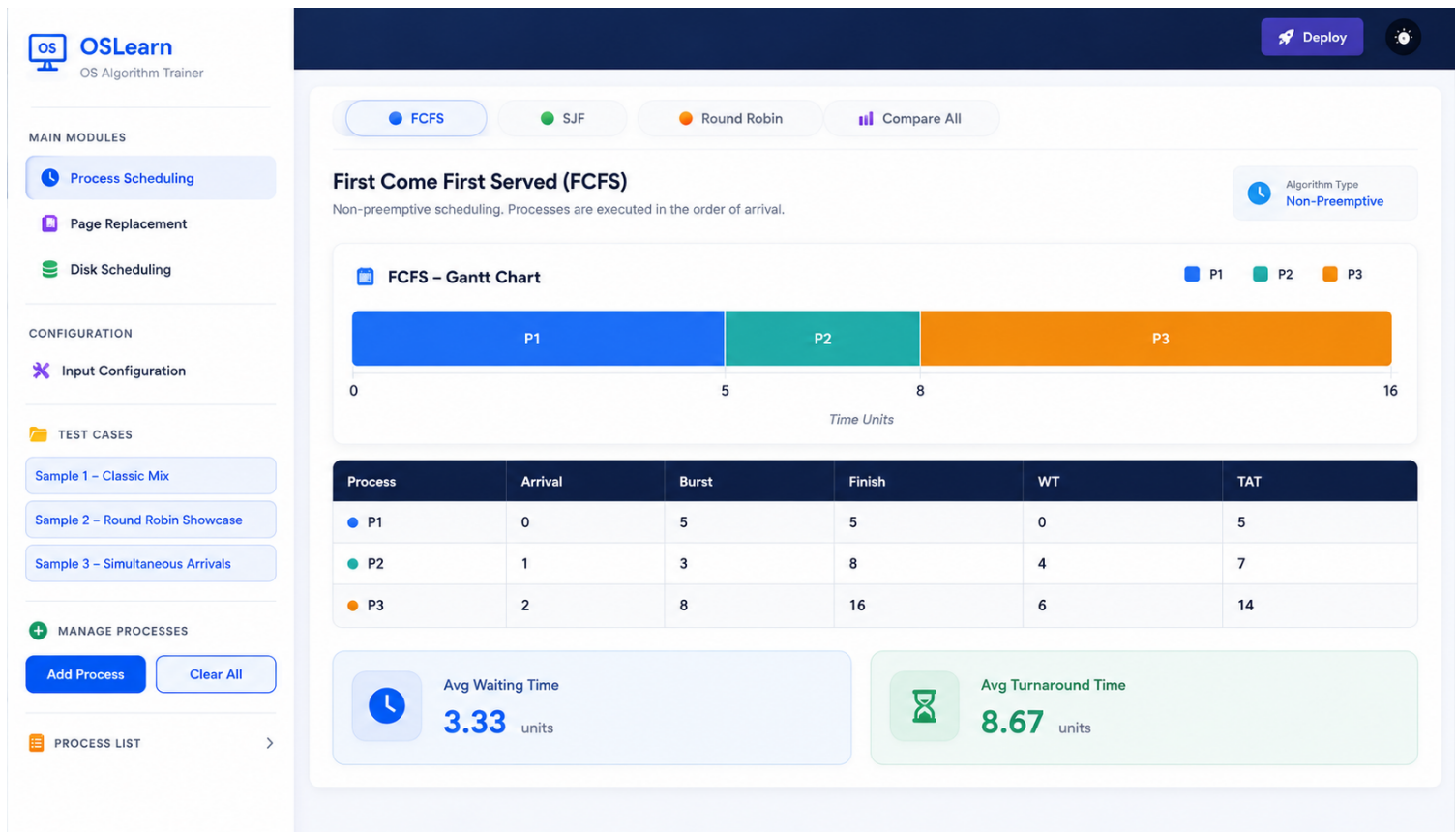
Reliability & Transparency: Every intermediate step (ready queues, frame contents, seek sequence) is shown so students can verify correctness.

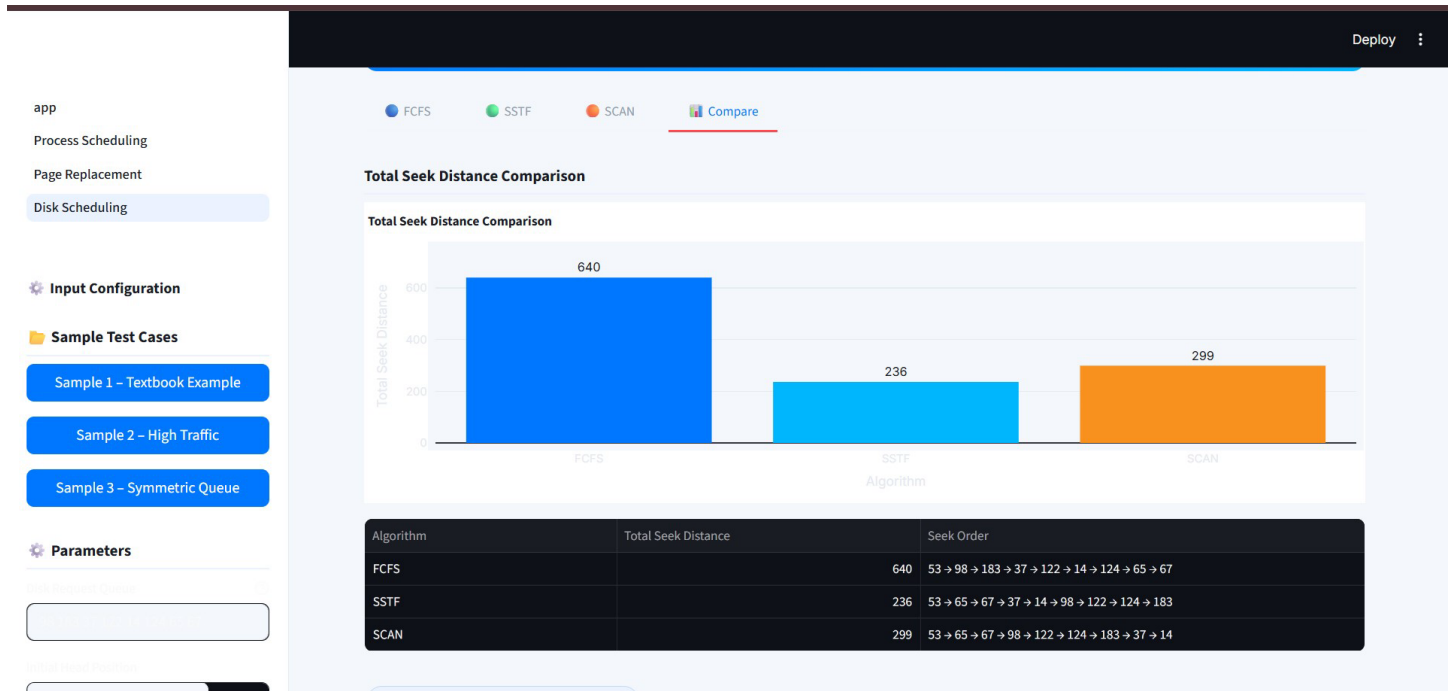
Sustainability: Pure in-memory Python execution; no cloud calls or heavy infrastructure—low energy footprint for classroom use.

Accessibility & Education Ethics: Distinct colour coding for faults vs hits, clear tables, and sample cases support diverse learners and reproducible assessment.

8. Conclusion

OSLearn successfully integrates Process Scheduling (FCFS, SJF, RR), Page Replacement (FIFO, LRU), and Disk Scheduling (FCFS, SSTF, SCAN) into a single Streamlit educational platform. Comparative metrics and step-by-step visualizations satisfy the functional, interface, performance, and responsible-computing requirements of Section D. Future extensions could include MLFQ, TLB visualization, and rotational-latency modelling.





9. Student Reflection

Integrating the three OS subsystems in one interactive application clarified how page faults trigger disk I/O and how the CPU scheduler reacts to blocked processes. Implementing the algorithms forced careful handling of edge cases (identical arrivals, empty queues, zero bursts) and reinforced the value of transparent intermediate state. With more time, live POSIX-thread demos or eBPF-based kernel tracing would further connect theory to production systems.

10. References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2014.
- [3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. New York, NY, USA: Pearson, 2017.
- [4] Streamlit Documentation, Multi-Page Applications and Session State, Snowflake Inc. [Online]. Available: <https://docs.streamlit.io>
- [5] Plotly Technologies Inc., Plotly Python Open Source Graphing Library. [Online]. Available: <https://plotly.com/python/>

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____